

Programming Language Design Concepts

Assoc.Prof. Tugba Onal-Suzek

Dr. Ozgur Kilic

2

Values and Types

- Types of values.
- Primitive, composite, recursive types.

Types (1)

- Values are grouped into **types** according to the operations that may be performed on them.
- Different PLs support different types of values (according to their intended application areas):
 - **C**: enumerands, integers, real numbers, structures, arrays, unions, pointers to variables, pointers to functions, char, byte
 - **Java**: booleans, integers, real numbers, arrays, objects, char, byte
 - **Haskell**: booleans, characters, integers, real numbers, tuples, disjoint unions, lists, recursive types.
 - **Perl** : scalars, arrays, hashes
 - **Python**: bool , int, tuples, list, dictionary, set, string,

Types (2)

- Roughly speaking, a **type** is a set of values:
 - v is a value of type T if $v \in T$.
- E is an *expression* of type T if E is guaranteed to yield a value of type T .
- But only certain sets of values are types:
 - $\{false, true\}$ is a type, since the operations *not*, *and*, and *or* operate uniformly over the values *false* and *true*.
 - $\{\dots, -2, -1, 0, +1, +2, \dots\}$ is a type, since operations such as addition and multiplication operate uniformly over all these values.
 - $\{13, true, Monday\}$ is not considered to be a type, since there are no useful operations over this set of values.

Types (3)

- More precisely, a **type** is a set of values, equipped with one or more **operations** that can be applied uniformly to all these values.
- The **cardinality** of a type T , written $\#T$, is the number of values of type T .

Primitive types

- A **primitive value** is one that cannot be decomposed into simpler values.
- A **primitive type** is one whose values are primitive.
- Every PL provides built-in primitive types.
- Some PLs also allow programs to define new primitive types.

Built-in primitive types (1)

- Typical built-in primitive types:

Boolean = $\{false, true\}$

Character = $\{..., 'A', ..., 'Z',$
 $..., '0', ..., '9',$
 $...\}$

PL- or implementation-defined
set of characters (ASCII, ISO-
Latin, or Unicode)

Integer = $\{..., -2, -1,$
 $0, +1, +2, ...\}$

PL- or implementation-defined
set of whole numbers

Float = $\{..., -1.0, ...,$
 $0.0, +1.0, ...\}$

PL- or implementation-defined
set of real numbers

- Names of types vary from one PL to another

Built-in primitive types (2)

- Cardinalities:

$\#Boolean = 2$

$\#Character = 128$ (ASCII), 256 (ISO-Latin), or 32768 (Unicode)

$\#Integer = \text{max integer} - \text{min integer} + 1$

- *Note:* In some PLs (such as C), booleans and characters are just small integers.

Defined primitive types

- In Java we can define new numeric types.
- In Java and C++ we can define new **enumeration types** simply by enumerating their values (called **enumerands**).

Example: Java enumerations

- Type declaration:

```
public enum Fruit { APPLE, PEAR, PEACH, PLUM};
```

- Here we defined a completely new type, FRUIT!!
- Set of values:

```
Fruit = {APPLE, PEAR, PEACH, PLUM}
```

- Fruit myfavoritefruit= PEAR;
- Cardinality:
#Fruit is 4

Example: C++ enumerations

- Type declaration:

```
typedef enum {red, green, blue} Color;
```

- Set of values:

Color = {*red, green, blue*}

- Cardinality:

#Color = 3

Composite types

- A **composite value** is one that is composed from simpler values.
- A **composite type** is a type whose values are composite.
- PLs support a huge variety of composite types.
- All these can be understood in terms of a few concepts:
 - Cartesian products (tuples, structures, records)
 - mappings (arrays)
 - disjoint unions (algebraic data types, discriminated records, objects)
 - recursive types (lists, trees, etc.)

Cartesian products (1)

- In a **Cartesian product**, values of several types are grouped into tuples.
- Let (x, y) stand for the **pair** whose first component is x and whose second component is y .
- Let $S \times T$ stand for the set of all pairs (x, y) such that x is chosen from set S and y is chosen from set T :

$$S \times T = \{ (x, y) \mid x \in S; y \in T \}$$

- Cardinality:

$$\#(S \times T) = \#S \times \#T$$

----- hence the “ $\times$ ” notation

Cartesian products (2)

- We can generalise from pairs to **tuples**. Let $S_1 \times S_2 \times \dots \times S_n$ stand for the set of all n -tuples such that the i th component is chosen from S_i :

$$S_1 \times S_2 \times \dots \times S_n = \{ (x_1, x_2, \dots, x_n) \mid x_1 \in S_1; x_2 \in S_2; \dots; x_n \in S_n \}$$

- Basic operations on tuples:
 - **construction** of a tuple from its component values
 - **selection** of an *explicitly-designated* component of a tuple.

so we can select the 1st or 2nd
(but not the i th) component

- **Records** (Ada), **structures** (C), and **tuples** (Haskell) can all be understood in terms of Cartesian products.

Example: C++ records (1)

- Type declarations:

```
typedef enum Month {jan, feb, mar, apr, may, jun, jul,  
aug, sep, oct, nov, dec} Month;  
typedef enum Day {1,2,3,4,5,6,7,8,9,...,31};  
struct Date{  
    Month m;  
    Day d;  
};
```

- Application code:

```
Date  someday = {ja  
...  
someday.d = 29;  
someday.m = feb;
```

record construction

component selection

Example: Haskell tuples

- Declarations:

```
data Month = Jan | Feb | Mar | Apr  
           | May | Jun | Jul | Aug  
           | Sep | Oct | Nov | Dec  
type Date = (Month, Int)
```

- Set of values:

Date = Month $\times$ Integer
 = {*Jan, Feb, ..., Dec*} $\times$ {..., -1, 0, 1, 2, ...}

- Application code:

```
someday = (jan, 1)  
m, d = someday  
anotherday = (m + 1, d)
```

tuple construction

component selection
(by pattern matching)

Week 2

Values and Types

- Types of values.
- Primitive, composite, recursive types.
- Type systems: static vs dynamic typing, type completeness.
- Expressions.
- Implementation notes.

Composite types

- A **composite value** is one that is composed from simpler values or **components**.
- A **composite type** is a type whose values are composite.
- PLs support a huge variety of composite types.
- All these can be understood in terms of a few concepts:
 - Cartesian products (tuples, structures, records)
 - mappings (arrays)
 - disjoint unions (algebraic data types, discriminated records, objects)
 - recursive types (lists, trees, etc.)

Mappings (1)

- We write $m : S \rightarrow T$ to state that m is a **mapping** from set S to set T . In other words, m maps every value in S to some value in T .
- If m maps value x to value y , we write $y = m(x)$. The value y is called the **image** of x under m .
- Some of the mappings in $\{u, v\} \rightarrow \{a, b, c\}$:

$$m_1 = \{u \rightarrow a, v \rightarrow c\}$$

$$m_2 = \{u \rightarrow c, v \rightarrow c\}$$

$$m_3 = \{u \rightarrow c, v \rightarrow b\}$$

image of u is c ,
image of v is b

Mappings (2)

- Let $S \rightarrow T$ stand for the set of all mappings from S to T :

$$S \rightarrow T = \{ m \mid x \in S \Rightarrow m(x) \in T \}$$

- What is the cardinality of $S \rightarrow T$?

There are $\#S$ values in S .

Each value in S has $\#T$ possible images under a mapping in $S \rightarrow T$.

So there are $\#T \times \#T \times \dots \times \#T$ possible mappings. Thus:

$$\#(S \rightarrow T) = (\#T)^{\#S}$$

$\#S$ copies of $\#T$ multiplied together

- For example, in $\{u, v\} \rightarrow \{a, b, c\}$ there are $3^2 = 9$ possible

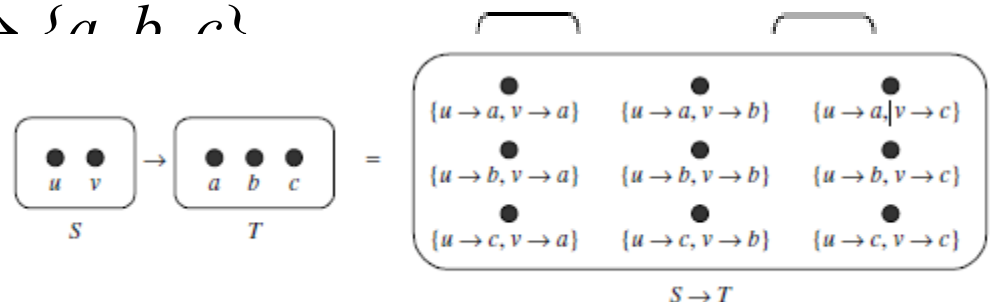


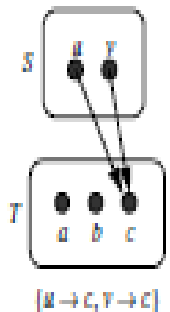
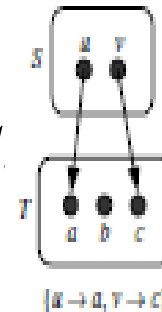
Figure 2.3 Set of all mappings in $S \rightarrow T$.

Arrays (1)

- **Arrays** (found in all imperative and OO PLs) can be understood as mappings.
- If an array's components are of type T and its index values are of type S , the array has one component of type T for each value in type S . Thus the array's type is $S \rightarrow T$.
- An array's **length** is the number of components, $\#S$.
- Basic operations on arrays:
 - **construction** of an array from its components
 - **indexing** – using a *computed* index value to select a component.
so we *can* select the *i*th component

Arrays (1)

- **Arrays** (found in all imperative and object oriented PLs) can be understood as mappings.
- If an array's components are of type T and its index values are of type S
 - the array has **one** component of type T **for each value in type S** .
 - Thus the array's type is $S \rightarrow T$.
- An array's **length** is the number of components, $\#S$.
- Basic operations on arrays:
 - **construction** of an array from its components
 - **indexing** – using a *computed* index value to select a component.



so we *can* select the i th component

Cardinality of Mappings(example)

- Consider the C++ declaration:

```
bool p[3];
```

- The set of all possible values of this array is

$$\{0, 1, 2\} \rightarrow \{false, true\}$$

The cardinality of this set of values is 2^3 , and the values are the following eight finite mappings:

$\{0 \rightarrow false, 1 \rightarrow false, 2 \rightarrow false\}$

$\{0 \rightarrow false, 1 \rightarrow false, 2 \rightarrow true\}$

$\{0 \rightarrow false, 1 \rightarrow true, 2 \rightarrow false\}$

$\{0 \rightarrow false, 1 \rightarrow true, 2 \rightarrow true\}$

$\{0 \rightarrow true, 1 \rightarrow false, 2 \rightarrow false\}$

$\{0 \rightarrow true, 1 \rightarrow false, 2 \rightarrow true\}$

$\{0 \rightarrow true, 1 \rightarrow true, 2 \rightarrow false\}$

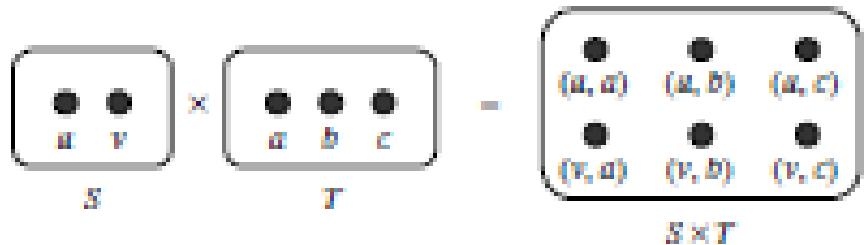
$\{0 \rightarrow true, 1 \rightarrow true, 2 \rightarrow true\}$

Cardinality of the Composite types

- Cartesian products (tuples, structures, records)

$S \times T$

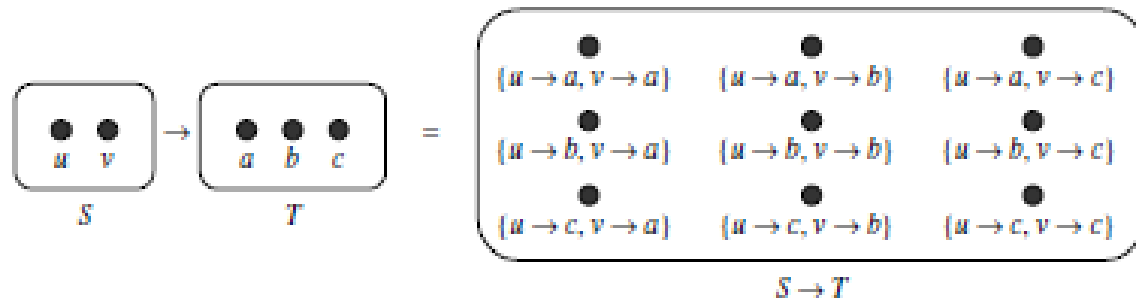
Cardinality is: $\#S \times \#T$



- mappings (arrays)

$S \rightarrow T$

Cardinality is: $(\#T)^{\#S}$



- disjoint unions (algebraic data types, discriminated records, objects)

- recursive types (lists, trees, etc.)

Arrays (2)

- An array of type $S \rightarrow T$ is a *finite* mapping.
- Here S is nearly always a finite range of consecutive values $\{l, l+1, \dots, u\}$. This is called the array's **index range**.
 - lower bound
 - upper bound
- In C and Java, the index range must be integers $\{0, 1, \dots, n-1\}$.
- How about multi-dimensional arrays?
 - If an array has index ranges of types $S_1, \dots, S_n$, the array's type is $S_1 \times \dots \times S_n \rightarrow T$.

Example: C++ arrays

- Type declarations:

```
bool HasEmployment[3];
```

- Application code:

```
HasEmployment[1]=false; indexing
```

```
int c=0;
```

```
HasEmployment[c] = !HasEmployment[c]; indexing
```

```
bool HasEmployment[] = {true, false, true}; array construction
```

Example: Perl arrays

- No type declarations in Perl and Python!
- Perl's Array construction:

`@mycolors = ('red', 'green', 'blue');` ----- array construction

- `$mycolors[1] = 'blue';` ----- indexing

A special array type in Perl: Hash (also called Associative Arrays)

For set of values

$$\text{Color} \rightarrow \text{Boolean} = \{\text{red}, \text{green}, \text{blue}\} \rightarrow \{0, 1\}$$

Define Hash type in Perl:

```
%Pixel= ( "red"=> 0, "green"=>1,"blue"=>0);
```

This is same as:

```
$Pixel{"red"}=0
```

```
$Pixel{"green"}=1
```

```
$Pixel{"blue"}=0
```

Cardinality of Pixel :

$$\#Pixel = (\#Boolean)^{\#Color} = 2^3 = 8$$

$\{red \rightarrow 0, green \rightarrow 0, blue \rightarrow 0\}$	$\{red \rightarrow 1, green \rightarrow 0, blue \rightarrow 0\}$
$\{red \rightarrow 0, green \rightarrow 0, blue \rightarrow 1\}$	$\{red \rightarrow 1, green \rightarrow 0, blue \rightarrow 1\}$
$\{red \rightarrow 0, green \rightarrow 1, blue \rightarrow 0\}$	$\{red \rightarrow 1, green \rightarrow 1, blue \rightarrow 0\}$
$\{red \rightarrow 0, green \rightarrow 1, blue \rightarrow 1\}$	$\{red \rightarrow 1, green \rightarrow 1, blue \rightarrow 1\}$